

MEMORY ORGANIZATION

Objective

This chapter introduces the concept of memory hierarchy, composed of cache memory, main memory, and auxiliary memory such as magnetic disks. The organization and operation of associative memory is explained. The concept of memory management is introduced through the presentation of hardware requirements for a cache memory and a virtual memory system.

OUTLINE

- 10.1 Memory Hierarchy
- 10.2 Main Memory
- 10.3 Auxiliary Memory
- 10.4 Associative Memory
- 10.5 Cache Memory
- 10.6 Virtual Memory
- 10.7 Memory Management Hardware

10.1 MEMORY HIERARCHY

Memory unit is essential component of digital computer since it is needed for storing programs and data. Memory unit that communicates directly with CPU is called Main memory. Devices that provide backup storage is called auxiliary memory. Only programs and data currently needed by processor reside in the main memory. All other information is stored in auxiliary memory and transferred to main memory when needed. The purpose of memory hierarchy is to obtain the highest possible access speed while minimizing the total cost of the memory system. The Figure 10.1 (a) and 10.1 (b) shows the organization of different kinds of memory and its hierarchy.

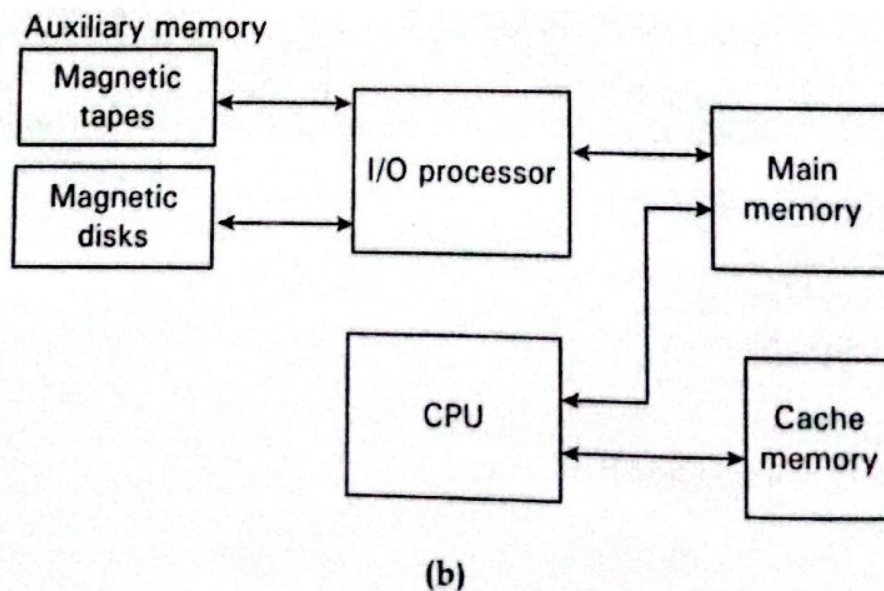
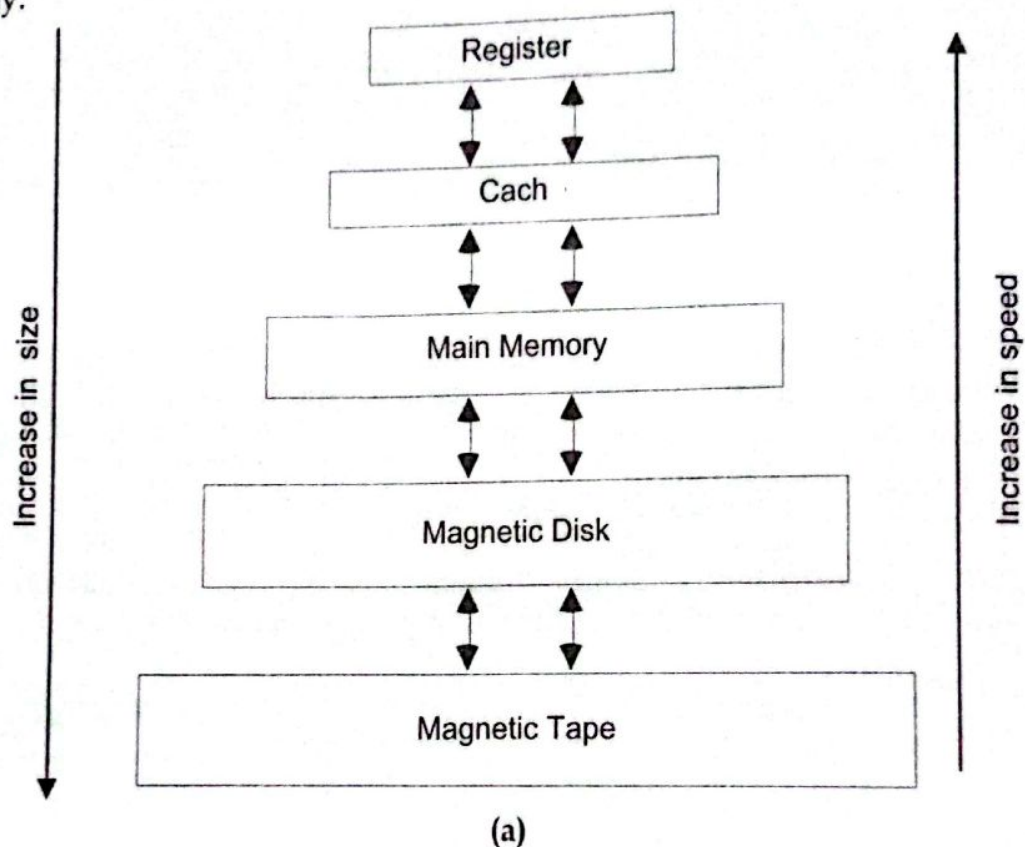


Fig 10.1 : Memory hierarchy in a computer system

Memory Address Map of RAM and ROM

The table, called a **memory address map**, is a pictorial representation of assigned address space for each chip in the system, shown in table 10.1. To demonstrate with a particular example, assume that a computer system needs 512 bytes of RAM and 512 bytes of ROM. The RAM and ROM chips to be used are specified in figure 10.1 and figure 10.2.

The diagram shows a rectangular block representing a 128x8 RAM module. On the left side, there are five input lines with labels: 'Chip select 1' connected to 'CS1', 'Chip select 2' connected to 'CS2', 'Read' connected to 'RD', 'Write' connected to 'WR', and '7-bit address' connected to 'AD 7'. On the right side, a double-headed horizontal arrow connects the module to the text '8 bit data bus'.



The diagram shows a 512x8 ROM chip with the following connections:

- Chip select 1 is connected to CS1.
- Chip select 2 is connected to CS2.
- A 9-bit address is connected to AD 9.
- The chip is connected to an 8-bit data bus.

Table 10.1 : Memory address map for microcomputer

[illegible]

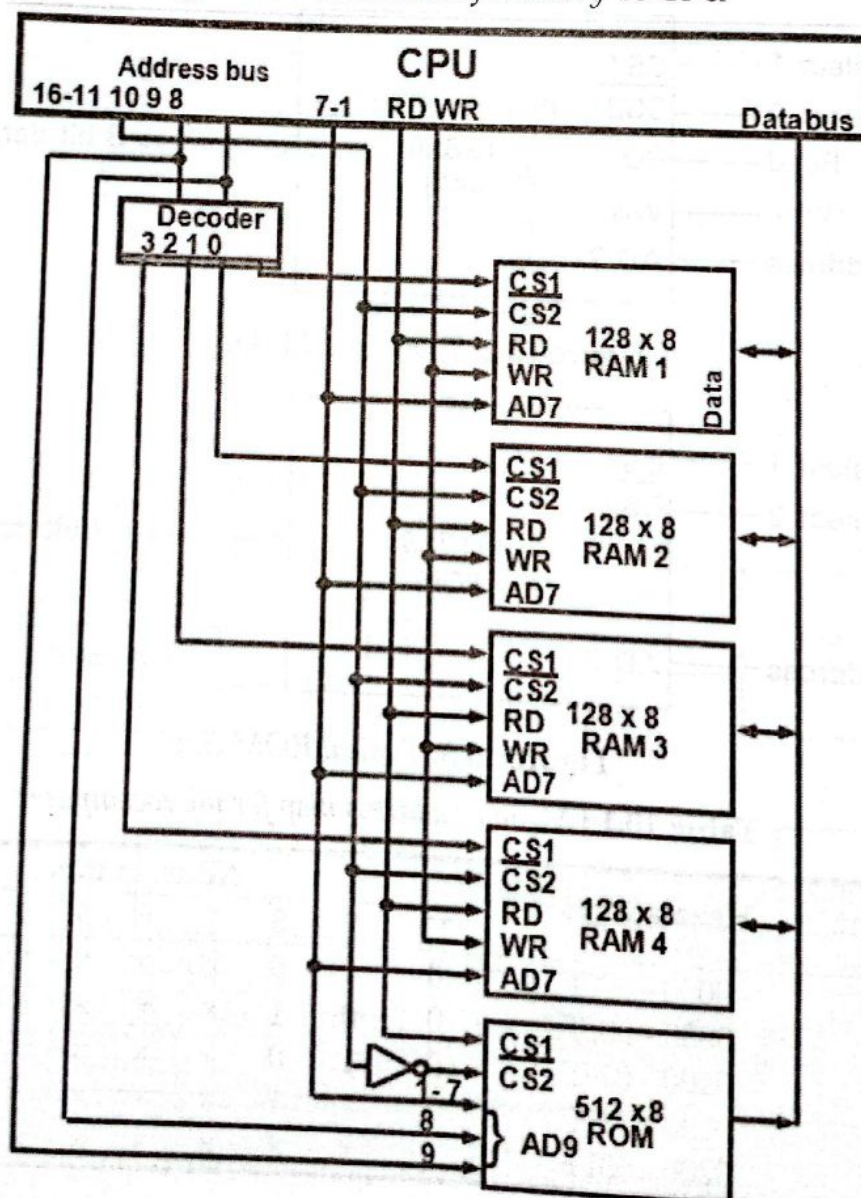
The component column specifies whether a RAM or a ROM chip is used. The hexadecimal address column assigns a range of hexadecimal equivalent addresses for each chip. The address bus lines are listed in the third column. Although there are 16 lines in the address bus, the table shows only 10 lines because the other 6 are not used in this example and are assumed to be zero. The small x's under the address bus lines designate those lines that must be connected to the address inputs in each chip.

The RAM chips have 128 bytes and need seven address lines. The ROM chip has 512 bytes and needs 9 address lines. The x's are always assigned to the low-order bus lines: lines 1 through 7 for the RAM and lines 1 through 9 for the ROM. It is now necessary to distinguish between four RAM chips by assigning to each a different address. For this particular example we choose bus lines 8 and 9 to represent four distinct binary combinations. The table clearly shows that the nine low-order bus lines constitute a memory space for RAM equal to $2^9 = 512$ bytes. The distinction between a RAM and ROM address is done with another bus line. Here we choose line 10 for this purpose. When line 10 is 0, the CPU selects a RAM, and when this line is equal to 1, it selects the ROM

Memory connections to CPU

RAM and ROM chips are connected to a CPU through the data and address buses. The low-order lines in the address bus select the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs.

Fig. 10.4: Connection of Memory To CPU



Magnetic Tape: Magnetic tapes are used for large computers like mainframe computers where large volume of data is stored for a longer time. In PC also you can use tapes in the form of cassettes. The cost of storing data in tapes is inexpensive. Tapes consist of magnetic materials that store data permanently. It can be 12.5 mm to 25 mm wide plastic film-type and 500 meter to 1200 meter long which is coated with magnetic material. The deck is connected to the central processor and information is fed into or read from the tape through the processor. It's similar to cassette tape recorder.

Magnetic tape is an information storage medium consisting of a magnetisable coating on a thin plastic strip. Nearly all recording tape is of this type, whether used for video with a video cassette recorder, audio storage (reel-to-reel tape, compact audio cassette, digital audio tape (DAT), digital linear tape (DLT) and other formats including 8-track cartridges) or general purpose digital data storage using a computer (specialized tape formats, as well as the above-mentioned compact audio cassette, used with home computers of the 1980s, and DAT, used for backup in workstation installations of the 1990s).

Magneto-optical and optical tape storage products have been developed using many of the same concepts as magnetic storage, but have achieved little commercial success.

Magnetic Disk: You might have seen the gramophone record, which is circular like a disk and coated with magnetic material. Magnetic disks used in computer are made on the same principle. It rotates with very high speed inside the computer drive. Data is stored on both the surface of the disk. Magnetic disks are most popular for direct access storage device. Each disk consists of a number of invisible concentric circles called tracks. Information is recorded on tracks of a disk surface in the form of tiny magnetic spots. The presence of a magnetic spot represents one bit and its absence represents zero bit. The information stored in a disk can be read many times without affecting the stored data. So the reading operation is non-destructive. But if you want to write a new data, then the existing data is erased from the disk and new data is recorded. For Example-Floppy Disk.

The primary computer storage device. Like tape, it is magnetically recorded and can be re-recorded over and over. Disks are rotating platters with a mechanical arm that moves a read/write head between the outer and inner edges of the platter's surface. It can take as long as one second to find a location on a floppy disk to as little as a couple of milliseconds on a fast hard disk. See hard disk for more details.

The disk surface is divided into concentric tracks (circles within circles). The thinner the tracks, the more storage. The data bits are recorded as tiny magnetic spots on the tracks. The smaller the spot, the more bits per inch and the greater the storage.

Sectors

Tracks are further divided into sectors, which hold a block of data that is read or written at one time; for example, READ SECTOR 782, WRITE SECTOR 5448. In order to update the disk, one or more sectors are read into the computer, changed and written back to disk. The operating system figures out how to fit data into these fixed spaces. Modern disks have more sectors in the outer tracks than the inner ones because the outer radius of the platter is greater than the inner radius

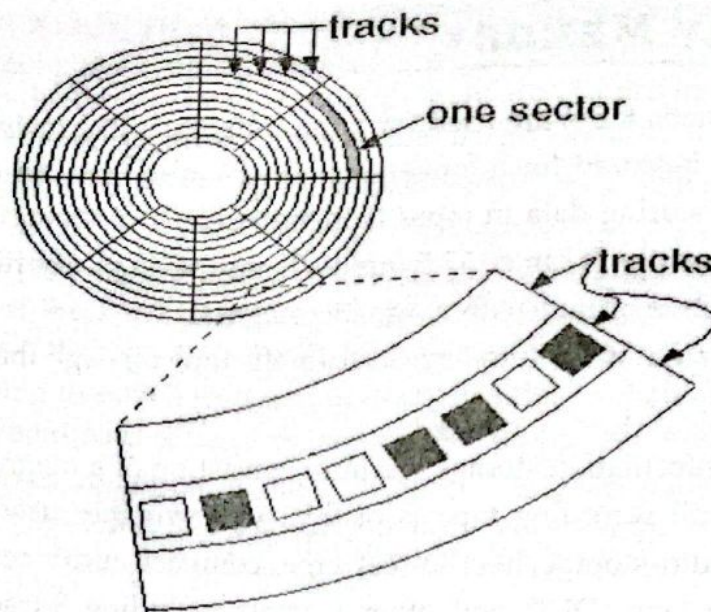


Figure 10.5: Block diagram of Magnetic Disk

Optical Disk: With every new application and software there is greater demand for memory capacity. It is the necessity to store large volume of data that has led to the development of optical disk storage medium. Optical disks can be divided into the following categories:

1. Compact Disk/ Read Only Memory (CD-ROM)
2. Write Once, Read Many (WORM)
3. Erasable Optical Disk

10.4 ASSOCIATIVE MEMORY: CONTENT ADDRESSABLE MEMORY (CAM)

The time required to find an item stored in memory can be reduced considerably if stored data can be identified for access by the content of the data itself rather than by an address. A memory unit accessed by content is called an associative memory or content addressable memory (CAM). This type of memory is accessed simultaneously and in parallel on the basis of data content rather than by specific address or location. The block diagram of an associative memory is shown in figure 10.6.

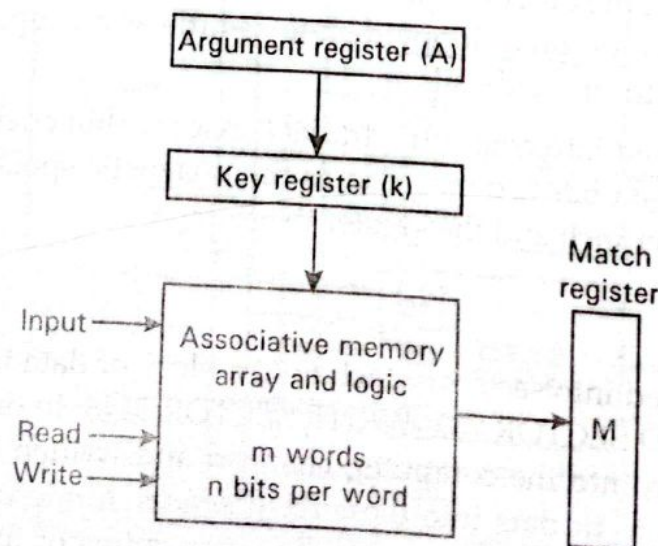


Figure 10.6: Block diagram of associative memory

It consists of a memory array and logic form words with n bits per word. The argument register A and key register K each have n bits, one for each bit of a word. The match register M has m bits, one for each memory word. Each word in memory is compared in parallel with the content of the argument register. The words that match the bits of the argument register set a corresponding bit in the match register. After the matching process, those bits in the match register that have been set indicate the fact that their corresponding words have been matched. Reading is accomplished by a sequential access to memory for those words whose corresponding bits in the match register have been set.

Hardware Organization

The key register provides a mask for choosing a particular field or key in the argument word. The entire argument is compared with each memory word if the key register contains all 1's. Otherwise, only those bits in the argument that have 1's in their corresponding position of the key register are compared. Thus the key provides a mask or identifying piece of information which specifies how the reference to memory is made. To illustrate with a numerical example, suppose that the argument register A and the key register K have the bit configuration shown below. Only the three leftmost bits of A are compared with memory words because K has 1's in these position. Word 2 matches the unmasked argument field because the three leftmost bits of the argument and the word are equal.

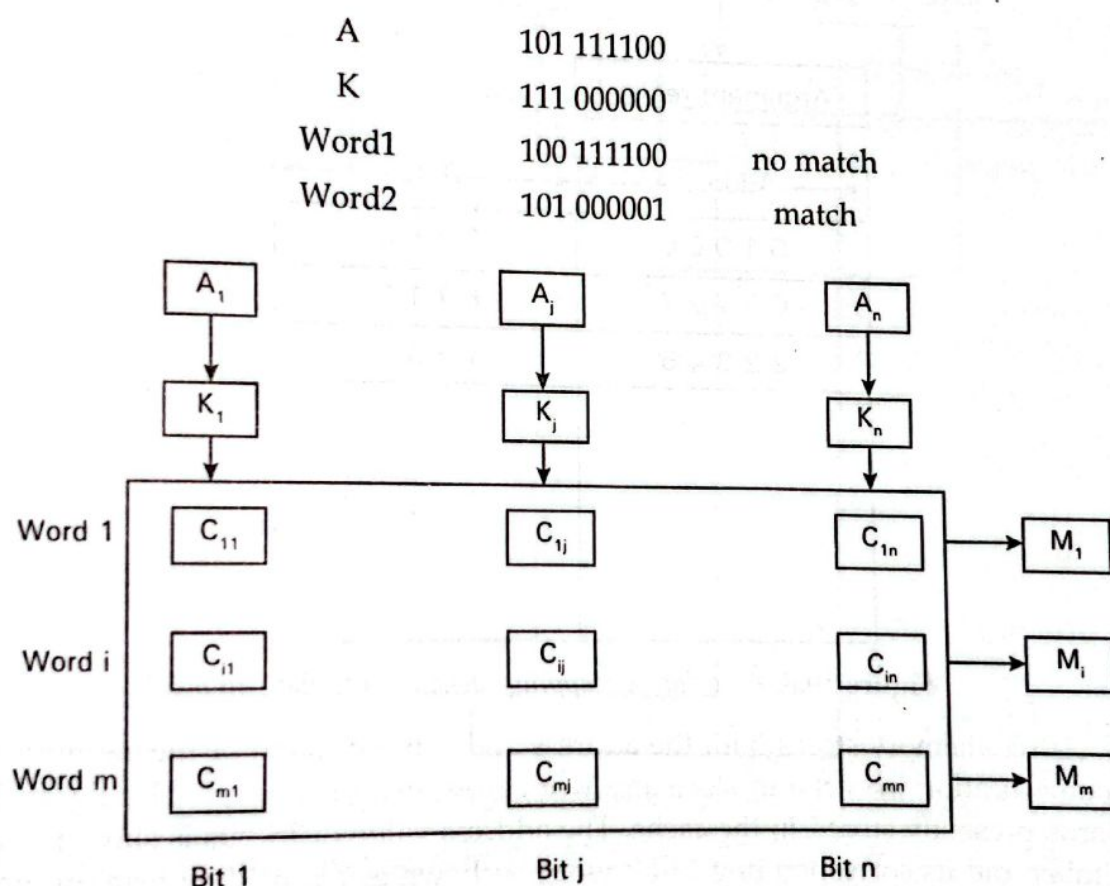


Figure 10.7: Associative Memory of m word, n cells per word.

The relation between the memory array and external registers in an associative memory is shown in figure 10.7. The cells in the array are marked by the letter C with two subscripts. The first subscript gives the word number and the second specifies the bit position in the word. Thus cell C_{ij} is the cell for bit j in words i . A bit A_j in the argument register is compared with all the bits in column j of the array provided that $K_j = 1$. This is done for all columns $j = 1, 2, \dots, n$. If a match occurs between all the unmasked bits of the argument and the bits in word i , the

corresponding bit M_i in the match register is set to 1. If one or more unmasked bits of the argument and the word do not match, M_i is cleared to 0.

10.5 CACHE MEMORY

Cache is a fast small capacity memory that should hold those information which are most likely to be accessed. The basic operation of the cache is, when the CPU needs to access memory, the cache is examined. If the word is found in the cache, it is read from the fast memory. If the word addressed by the CPU is not found in the cache, the main memory is accessed to read the word. The transformation of data from main memory to cache memory is referred to as a **mapping process**.

Associative mapping

Consider the main memory can store 32K words of 12 bits each. The cache is capable of storing 512 of these words at any given time. For every word stored in cache, there is a duplicate copy in main memory. The CPU communicates with both memories. It first sends a 15-bit address to cache. If there is a hit, the CPU accepts the 12-bit data from cache, if there is miss, the CPU reads the word from main memory and the word is then transferred to cache.

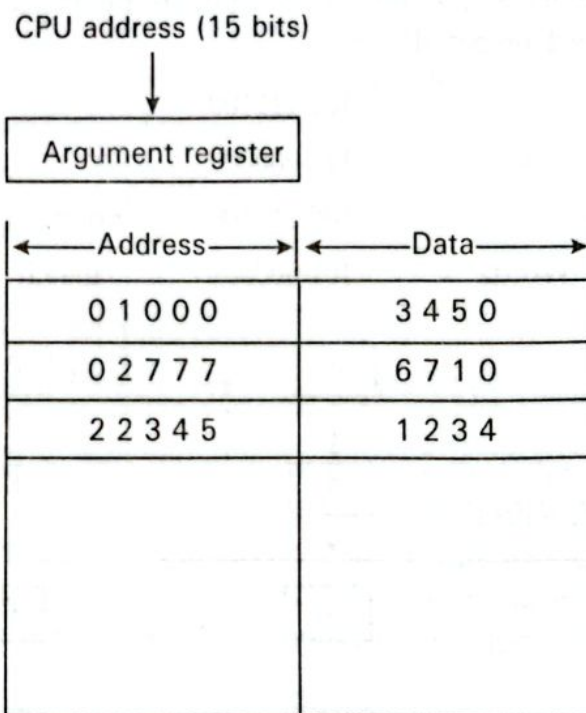


Figure 10.8: Associative mapping cache (all numbers in octal)

The associative memory stores both the address and content (data) of the memory word. This permits any location in cache to store any word from main memory. The figure 10.8 shows three words presently stored in the cache. The address value of 15 bits is shown as a five-digit octal number and its corresponding 12-bit word is shown as a four-digit octal number. A CPU address of 15 bits is placed in the argument register and the associative memory is searched for a matching address. If the address is found the corresponding 12-bit data is read and sent to CPU. If no match occurs, the main memory is accessed for the word. The address data pairs then transferred to the associative cache memory. If the cache is full, an address data pair must be displaced to make room for a pair that is needed and not presently in the cache. This constitutes a first-in first-one (FIFO) replacement policy.

Direct Mapping in Organization of Cache Memory

The CPU address of 15 bits is divided into two fields. The nine least significant bits constitute the index field and the remaining six bits from the tag field. The figure 10.9 shows that main memory needs an address that includes both the tag and the index.

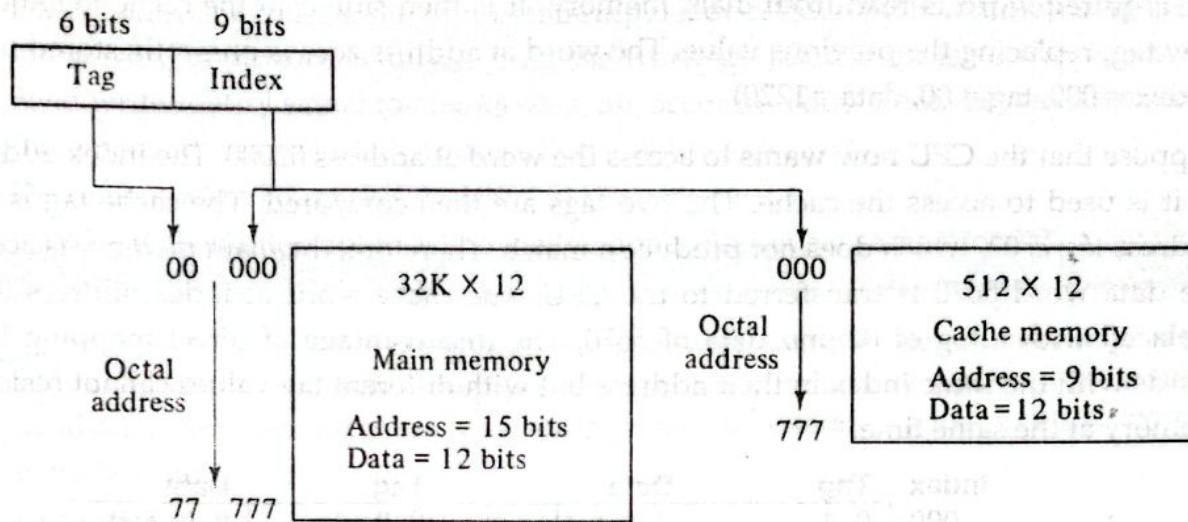


Figure 10.9: Addressing relationships between main and cache memories

The number of bits in the index field is equal to the number of address bits required to access the cache memory. The internal organization of the words in the cache memory is as shown in figure 10.9.

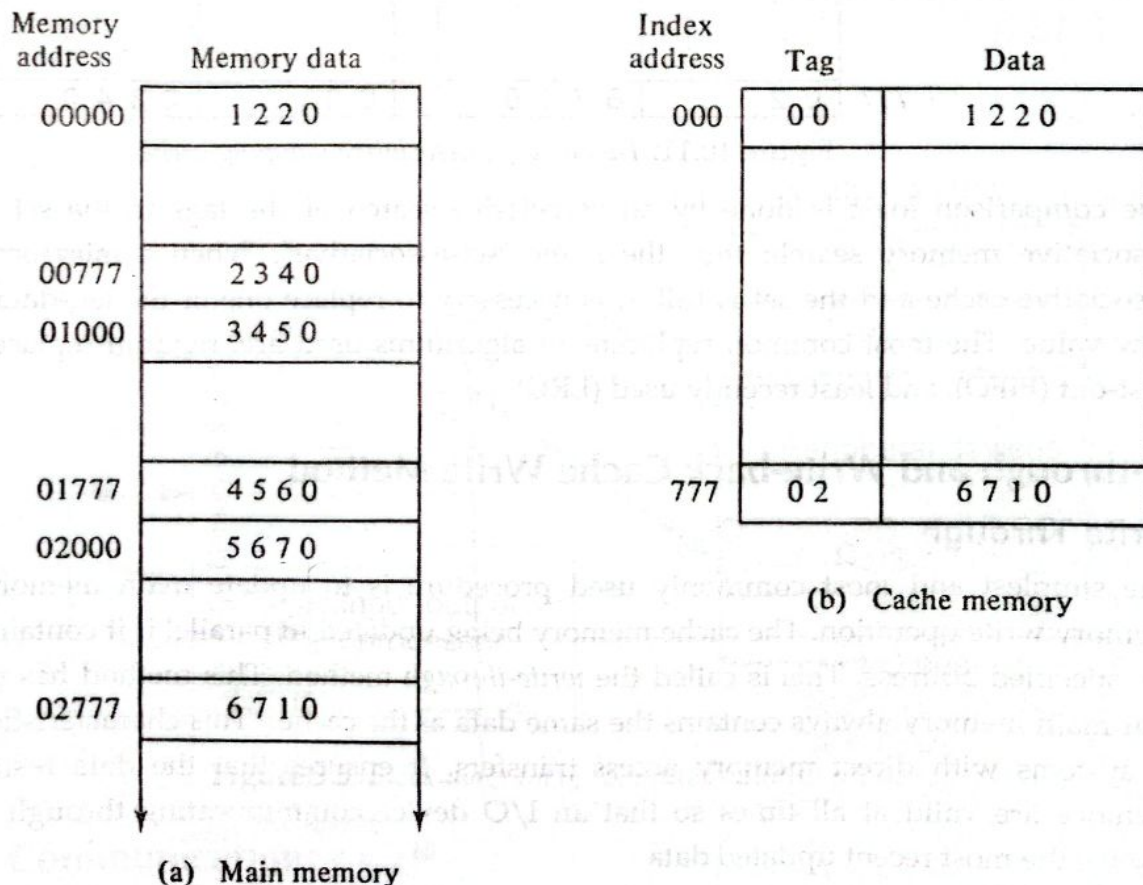


Figure 10.10: Direct mapping cache organization

Each word in cache consists of the data word and its associated tag. When a new word is first brought into the cache, the tag bits are stored alongside the data bits. When the CPU generates a memory request the index field is used for the address to access the cache. The tag field of the CPU address is compared with the tag in the word read from the cache. If the two tags match, there is a hit and the desired data word is in cache and if there is no match, there is a miss and the required word is read from main memory. It is then stored in the cache together with the new tag, replacing the previous value. The word at address zero is presently stored in the cache (index = 000, tag = 00, data = 1220).

Suppose that the CPU now wants to access the word at address 02000. The index address is 000, so it is used to access the cache. The two tags are then compared. The cache tag is 00 but the address tag is 02, which does not produce a match. Therefore, the main memory is accessed and the data word 5670 is transferred to the CPU. The cache word at index address 000 is then replaced with a tag of 02 and data of 5670. The **disadvantage** of direct mapping is that two words with the same index in their address but with different tag values cannot reside in cache memory at the same time.

Index	Tag	Data	Tag	Data
000	0 1	3 4 5 0	0 2	5 6 7 0
777	0 2	6 7 1 0	0 0	2 3 4 0

Figure 10.11: Two way set associative mapping cache

The comparison logic is done by an associative search of the tags in the set similar to an associative memory search: thus the name "set-associative". When a miss occurs in a set-associative cache and the set is full, it is necessary to replace one of the tag-data items with a new value. The most common replacement algorithms used are: random replacement, first-in first-out (FIFO), and least recently used (LRU).

Write-through and Write-back Cache Write Method

Write Through

The simplest and most commonly used procedure is to update main memory with every memory write operation. The cache memory being updated in parallel if it contains the word at the specified address. This is called the *write-through* method. This method has the advantage that main memory always contains the same data as the cache. This characteristic is important in systems with direct memory access transfers. It ensures that the data residing in main memory are valid at all times so that an I/O device communicating through DMA would receive the most recent updated data.

Write-Back (Copy-Back)

The second procedure is called the write-back method. In this method only the cache location is updated during a write operation. The location is then marked by a flag so that later when the word is removed from the cache it is copied into main memory. The reason for the write-back method is that during the time a word resides in the cache, it may be updated several times. However, as long as the word remains in the cache, it does not matter whether the copy in main memory is out of date, since requests from the word are filled from the cache. It is only when the word is displaced from the cache that an accurate copy need be rewritten into main memory.

10.6 VIRTUAL MEMORY

Virtual memory is used to give programmers the illusion that they have a very large memory at their disposal, even though the computer actually has a relatively small main memory. This system provides a mechanism for translating program-generated addresses into correct main memory locations.

Address space

An address used by a programmer will be called a virtual address, and the set of such addresses is known as address space.

Memory space

An address in main memory is called a location or physical address. The set of such locations is called the memory space.

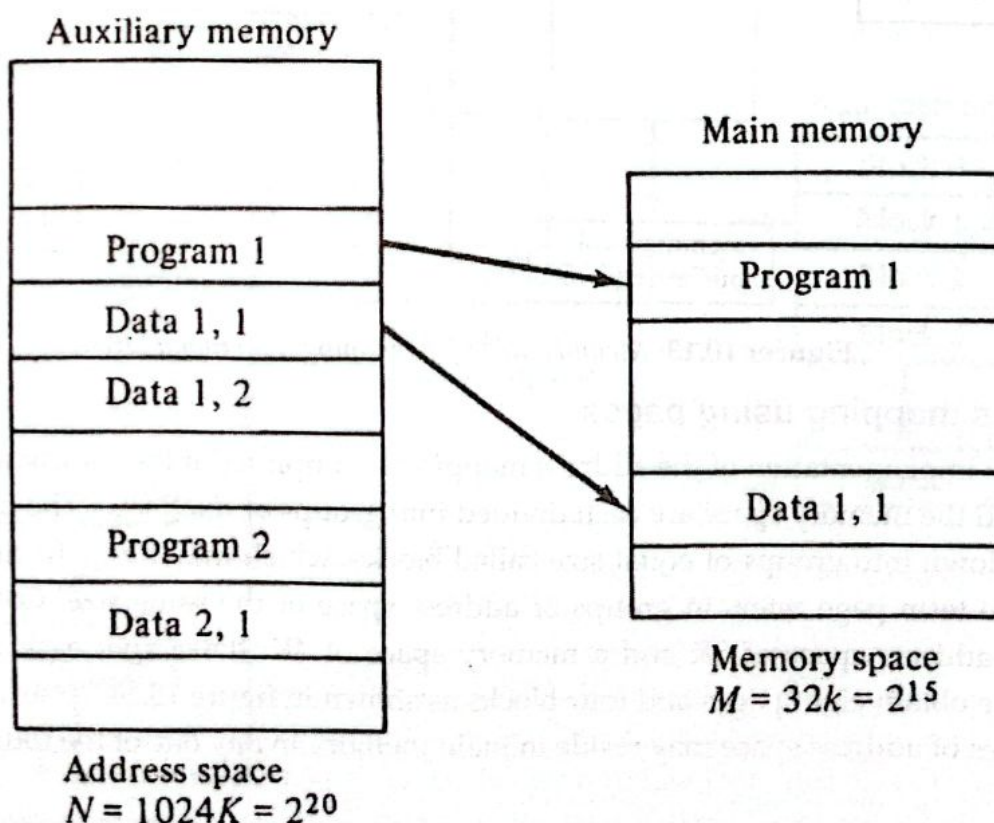
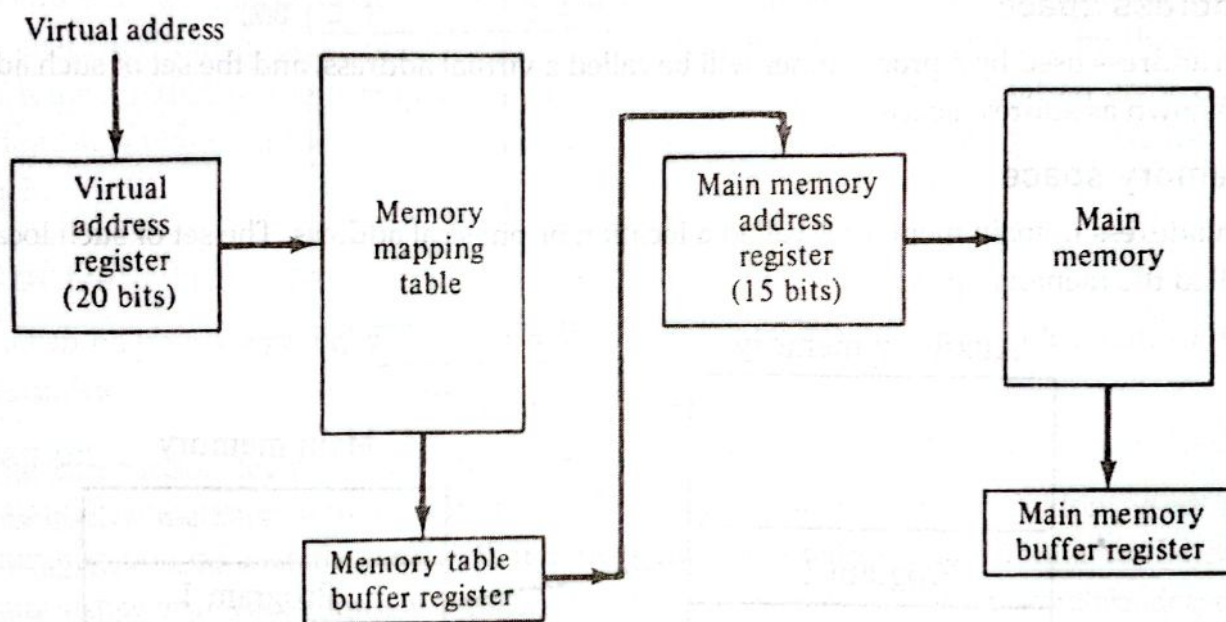


Figure 10.12: Relation between address and memory space in a virtual memory system

As an illustration, consider a computer with a main-memory capacity of 32K words ($K = 1024$). Fifteen bits are needed to specify a physical address in memory since $32K = 2^{15}$. Suppose that the computer has available auxiliary memory for storing $2^{20} = 1024K$ words. Thus auxiliary memory has a capacity for storing information equivalent to the capacity of 32 main memories.

Denoting the address space by N and the memory space by M , we then have for this example $N = 1024K$ and $M = 32K$. In a multiprogramming computer system, programs and data are transferred to and from auxiliary memory and main memory based on demands imposed by the CPU. Suppose that program 1 is currently being executed in the CPU. Program 1 and a portion of its associated data are moved from auxiliary memory into main memory as shown in figure 10.12. Portions of programs and data need not be in contiguous locations in memory since information is being moved in and out, and empty spaces may be available in scattered locations in memory. In our example, the address field of an instruction code will consist of 20 bits but physical memory addresses must be specified with only 15 bits. Thus CPU will reference instructions and data with a 20-bit address, but the information at this address must be taken from physical memory because access to auxiliary storage for individual words will be too long.



Figurer 10.13: Memory table for mapping a virtual address

Address mapping using pages

The table implementation of the address mapping is simplified if the information in the address space and the memory space are each divided into groups of fixed size. The physical memory is broken down into groups of equal size called blocks, which may range from 64 to 4096 words each. The term page refers to groups of address space of the same size. Consider a computer with an address space of 8K and a memory space of 4K. If we split each into groups of 1K words we obtain eight pages and four blocks as shown in figure 10.14. At any given time, up to four pages of address space may reside in main memory in any one of the four blocks.

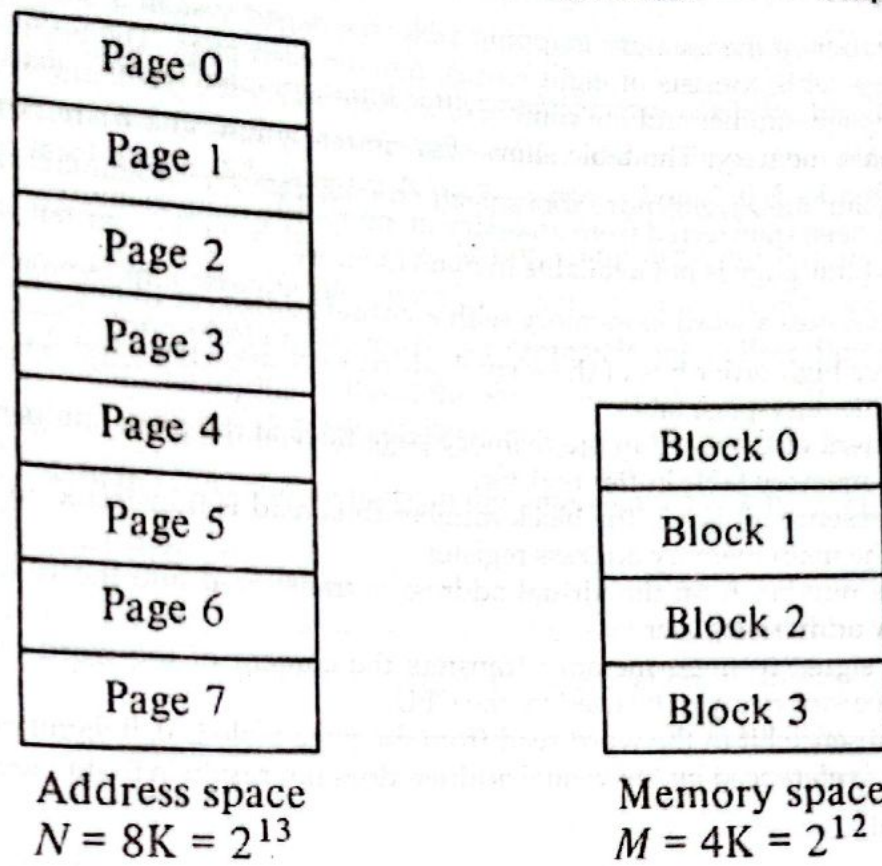


Figure 10.14: Address and Memory

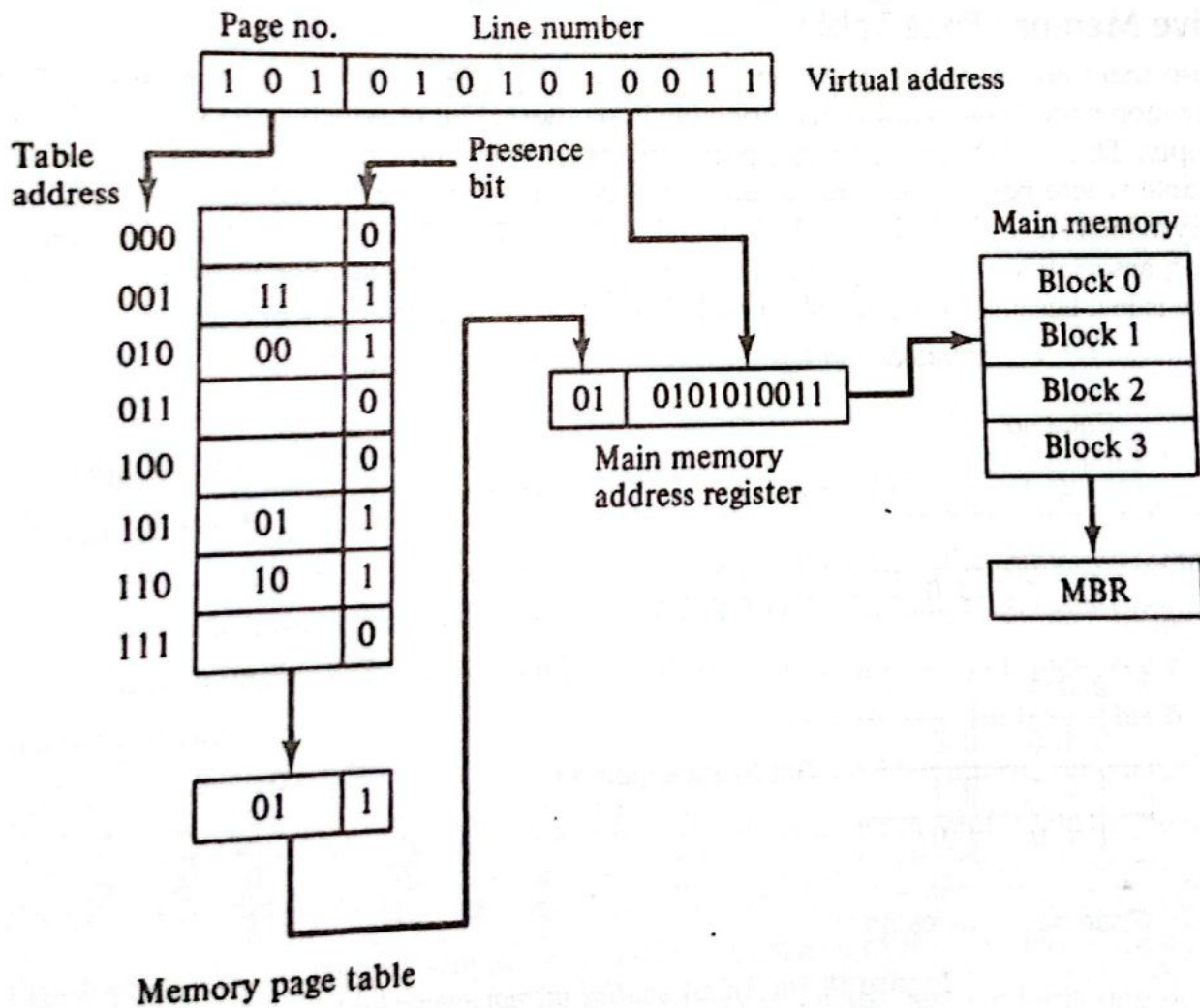


Figure 10.15: Memory table in paged system

The organization of the memory mapping table in a paged system is shown in figure 10.15. The memory-page table consists of eight words, one for each page. The address in the page table denotes the page number and the content of the word give the block number where that page is stored in main memory. The table shows that pages 1, 2, 5, and 6 are now available in main memory in blocks 3, 0, 1, and 2, respectively. A presence bit in each location indicates whether the page has been transferred from auxiliary memory into main memory. A 0 in the presence bit indicates that this page is not available in main memory.

The CPU references a word in memory with a virtual address of 13 bits.

- The three high-order bits of the virtual address specify a page number and also an address for the memory-page table.
- The content of the word in the memory page table at the page number address is read out into the memory table buffer register.
- If the presence bit is a 1, the block number thus read is transferred to the two high-order bits of the main memory address register.
- The line number from the virtual address is transferred into the 10 low-order bits of the memory address register.
- A read signal to main memory transfers the content of the word to the main memory buffer register ready to be used by the CPU.
- If the presence bit in the word read from the page table is 0, it signifies that the content of the word referenced by the virtual address does not reside in main memory.

Page Fault

If a page number cannot be found in the Page Table it is called page fault

Associative Memory Page Table

Suppose there are m blocks in memory n number of pages in virtual address space. In this organization page Table requires n entry table in memory. Out of which $n-m$ entries of the table are empty. This is inefficient storage space utilization. More efficient method is use a m -entry page table where page table is made up of an Associative Memory. Associative memory is a memory which stores both data and it's actual address. There fore if page table is implemented by using associative memory it stores both page no. and it's associated block no. associative memory is fast because here normally parallel search is done on the basis of content.

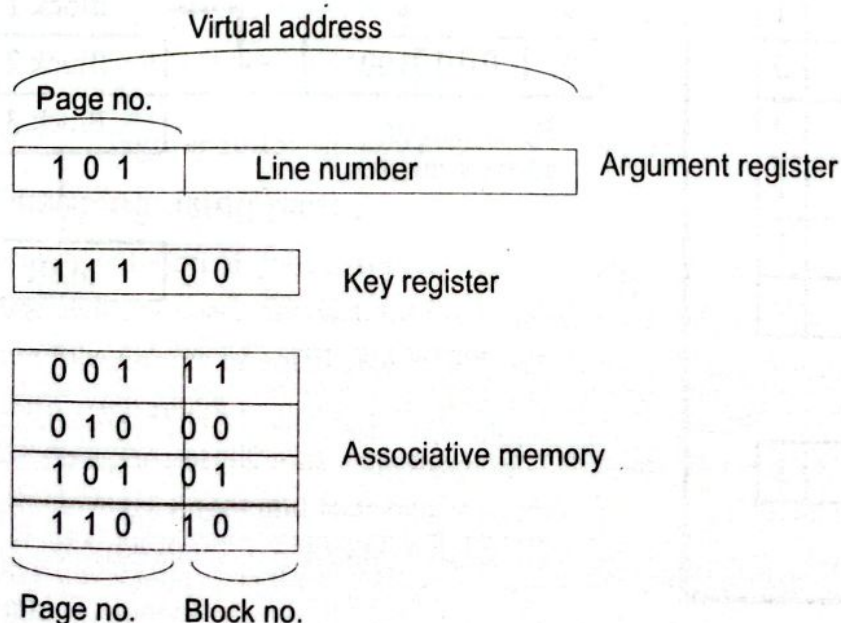


Figure 10.16: An associative memory page table

Here key register is used for masking. That is we need to match only first three bits

Page Replacement

Page Replacement determines which page to displace from memory to make room for an incoming page when no free frame is available

- Find the location of the desired page on the backing store (secondary storage)
- Find a free frame
 - ▲ If there is a free frame, use it
 - ▲ Otherwise, use a page-replacement algorithm to select a victim frame
 - ▲ Write the victim page to the backing store
- Read the desired page into the (newly) free frame
- Restart the user process

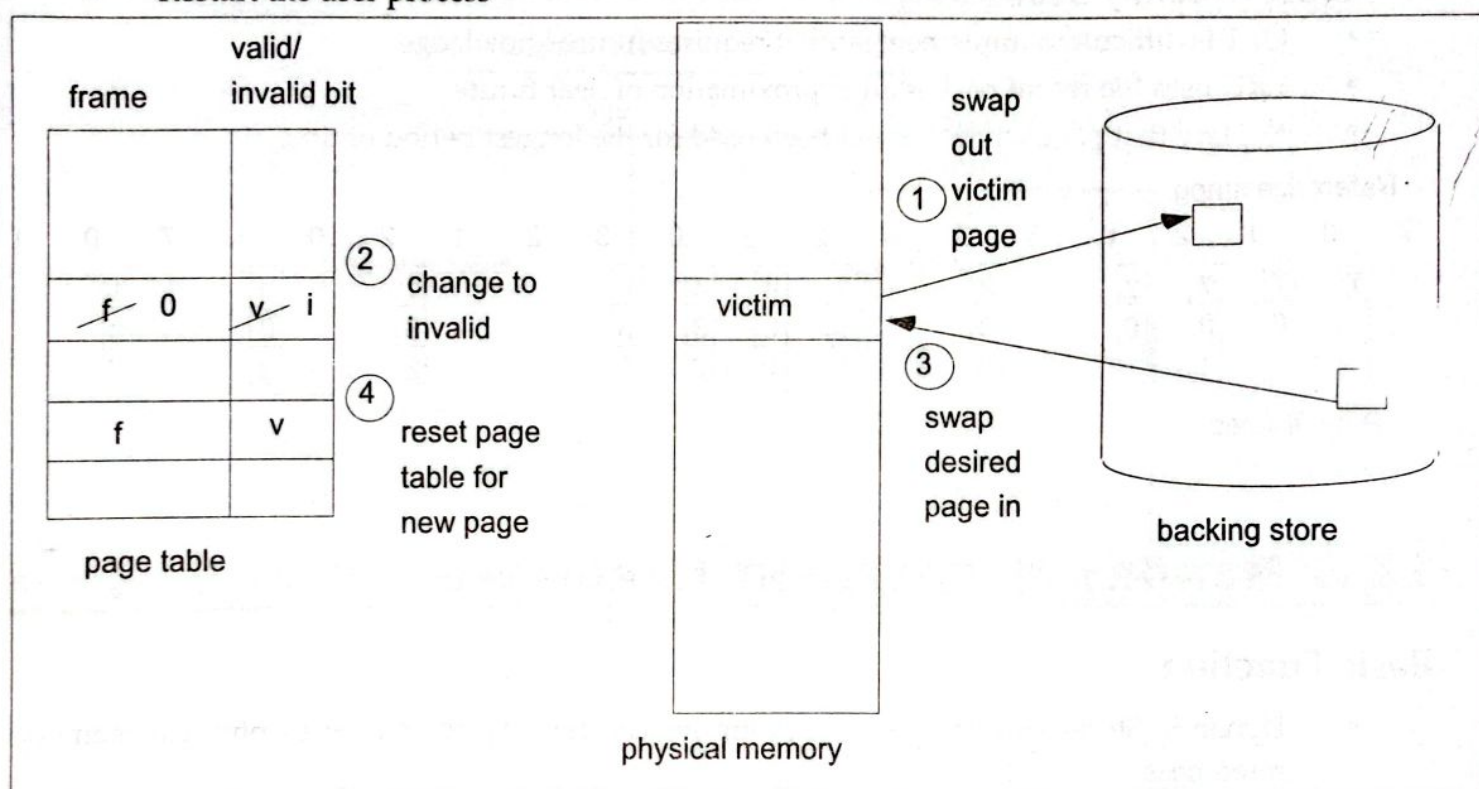
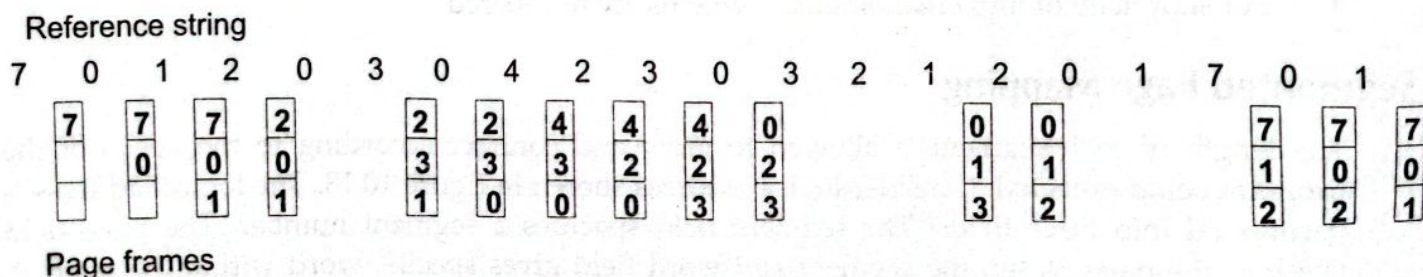


Figure 10.17 Page replacement process

Page Replacement algorithms

First In First Out (FIFO)

- FIFO algorithm selects the page that has been in memory the longest time using a queue
- Every time a page is loaded, its identification is inserted in the queue
- Easy to implement:: may result in a frequent page fault



Optimal Replacement (OPT) –

- Lowest page fault rate of all algorithms
- Replace that page which will not be used for the longest period of time

Reference string

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2		2			2				2			7		
	0	0	0		0		4			0				0			0		
		1	1		3		3			3				1			1		

Page frames

Least Recently Used (LRU)

- OPT is difficult to implement since it requires future knowledge
- LRU uses the recent past as an approximation of near future.
- Replace that page which has not been used for the longest period of time

Reference string

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2		4	4	4	0				1		1			
	0	0	0		0		0	0	3	3				3		0			
		1	1		3		3	2	2	2				2		2			7

Page frames

10.7 MEMORY MANAGEMENT HARDWARE

Basic Function

- Dynamic Storage Relocation - mapping logical memory references to physical memory references
- Provision for sharing common information stored in memory by different users
- Protection of information against unauthorized access

Segmentation

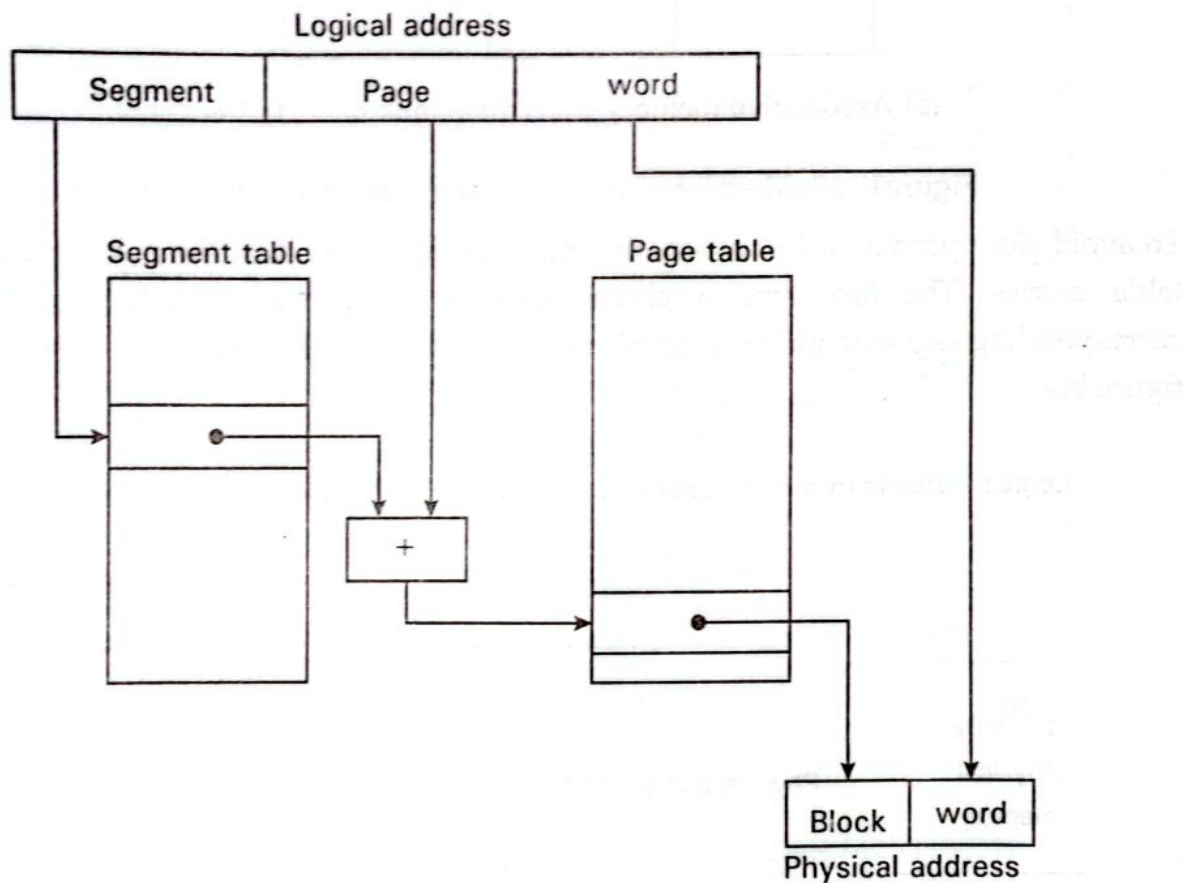
- A memory management scheme where logical address space is a collection of segments
- Each segment has a name and a length
- Address specifies both the segment name and the offset within the segment.
- For simplicity of implementations, segments are numbered.

Segmented Page Mapping

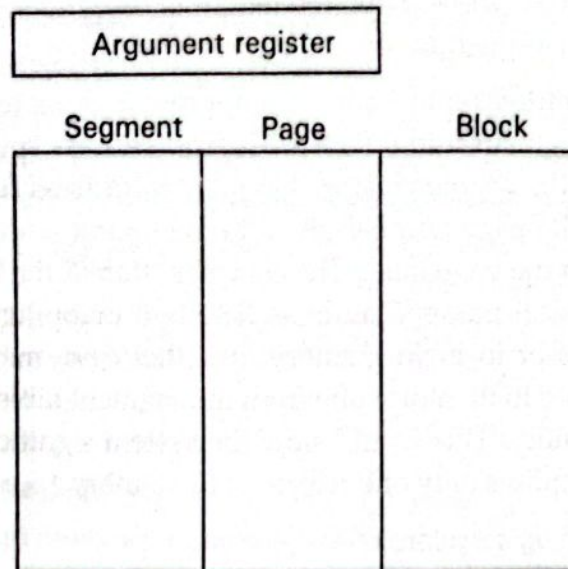
The length of each segment is allowed to grow and contract according to the needs of the program being executed. Consider logical address shown in figure 10.18. The logical address is partitioned into three fields. The segment field specifies a segment number. The page field specifies the page within the segment and word field gives specific word within the page. A page field of k bits can specify up to 2^k pages. A segment number may be associated with just

one page or with as many as 2k pages. Thus the length of a segment would vary according to the number of pages that are assigned to it.

The mapping of the logical address into a physical address is done by means of two tables, as shown in figure 10.18. The segment number of the logical address specifies the address for the segment table. The entry in the segment table is a pointer address for a page table base. The page table base is added to the page number given in the logical address. The sum produces a pointer address to an entry in the page table. The concatenation of the block field with the word field produces the final physical mapped address. The two mapping tables may be stored in two separate small memories or in main memory. In either case, memory reference from the CPU will require three accesses to memory: one from the segment table, one from the page table and the third from main memory. This would slow the system significantly when compared to a conventional system that requires only one reference to memory.



(a) Logical to physical address mapping

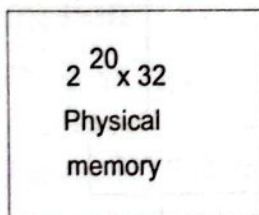


(b) Associative memory translation look-aside buffer (TLB)

Figure 10.18 : Mapping of segmented-page memory management unit

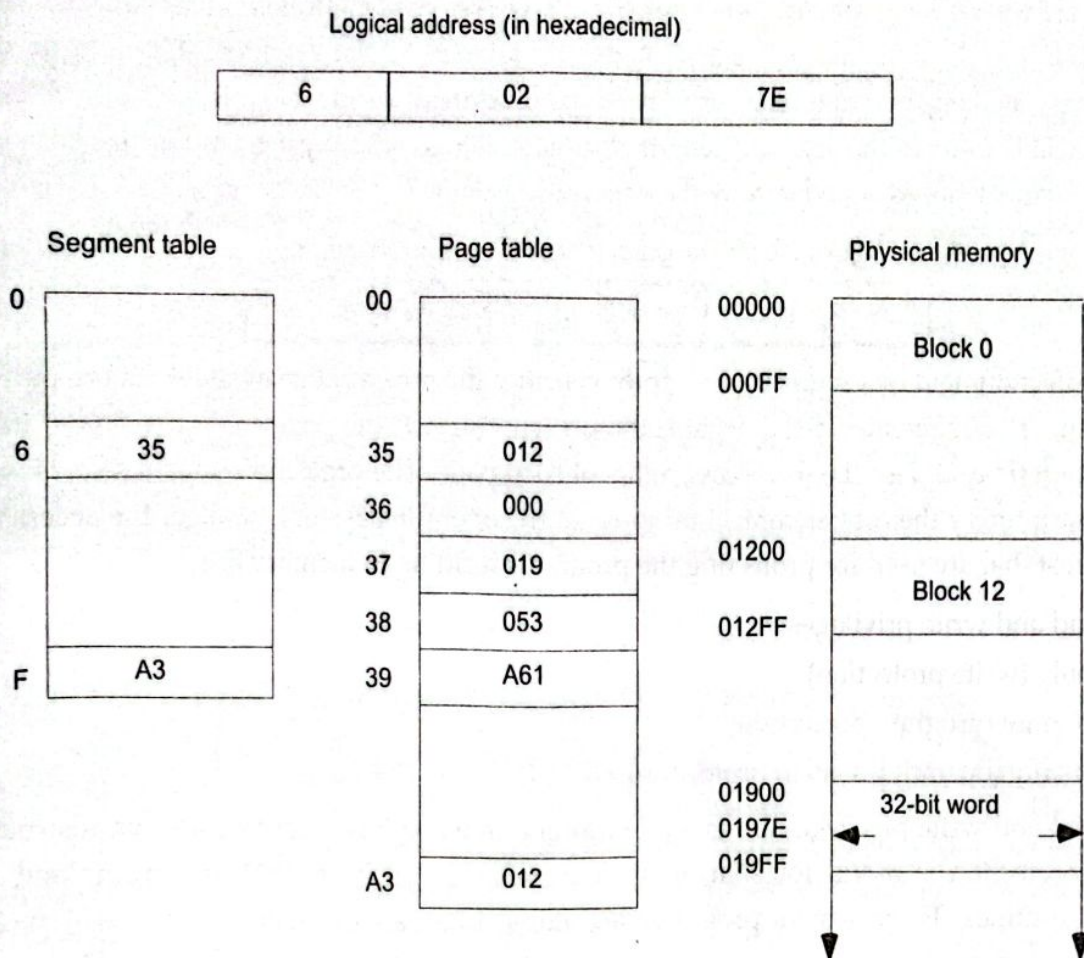
To avoid slow process, a fast associative memory is used to hold the most recently referenced table entries. The first time a given block is referenced, its value together with the corresponding segment and page numbers are entered into the associative memory as shown in figure b.

Logical address format: 16 segments of 256 pages each, each page has 256 words



Physical address format: 4096 blocks of 256 words each, each word has 32bits

**Figure 10.19:** An example of Logical and Physical Address

Example:

Physical address = Page table [page no. + segment table [segment no]] + line no (word)

[x] → Content at location x

Figure 10.20 : Logical to physical address mapping example

Memory Protection

Memory protection can be assigned to the physical address or the logical address. The protection of memory through the physical address can be done by assigning to each block in memory a number of protection bits that indicate the type of access allowed to its corresponding block. Every time a page is moved from one block to another it would be necessary to update the block protection bits. A much better place to apply protection is in the logical address space rather than the physical address space. This can be done by including protection information within the segment table or segment register of the memory management hardware.

The content of each entry in the segment table or a segment register is called a descriptor. A typical descriptor would contain, in addition to a base address field, one or two additional fields for protection purposes. A typical format for a segment descriptor is shown in Fig. 12-25. The base address field gives the base of the page table address in a segmented-page

organization or the block base address in a segment register organization. This is the address used in mapping from a logical to the physical address. The length field gives the segment size by specifying the maximum number of pages assigned to the segment. The length field is compared against the page number is the logical address. A size violation occurs if the page number falls outside the segment length boundary. Thus a given program and its data cannot access memory not assigned to it by the operating system.

Figure 10.21: *Format of a typical segment descriptor*

Base address	Length	Protection
--------------	--------	------------

The protection field in a segment descriptor specifies the access rights available to the particular segment. In a segmented-page organization, each entry in the page table may have its own protection field to describe the access rights of each page. The protection information is set into the descriptor by the master control program of the operating system. Some of the access rights of interest that are used for protecting the programs residing in memory are:

1. Full read and write privileges
2. Read only (write protection)
3. Execute only (program protection)
4. System only (operating system protection)

Full read and write privileges are given to a program when it is executing its own instructions. Write protection is useful for sharing system programs such as utility programs and other library routines. These system programs are stored in an area of memory where they can be shared by many users. They can be read by all programs, but no writing is allowed. This protects them from being changed by other programs.

The execute-only condition protects programs from being copied. It restricts the segment to be referenced only during the instruction fetch phase but not during the execute phase. Thus it allows the users to execute the segment program instructions but prevents them from reading the instructions as data for the purpose of copying their content.

Portions of the operating system will reside in memory at any given time. These system programs must be protected by making them inaccessible to unauthorized users. The operating system protection condition is placed in the descriptors of all operating system programs to prevent the occasional user from accessing operating system segments.

